MLDS Assignment 1

From initial policy-gradient failures to controlled improvements
Complete report | 23 October 2026

GitHub username: Dorrie-ge
Repository: mlds-assignment1-rl
https://github.com/Dorrie-ge/mlds-assignment1-rl

Main findings

The shuttle MDP has a dispatch threshold of 13 passengers; value iteration and policy iteration agree. CartPole learning improves after reducing the original Adam learning rate, with considerable variation between training seeds. Pong initially remains near a -21 game score. Extending the same configuration, changing actor time weights and changing image representation produce only modest gains. A matched actor-learning-rate comparison then produces a much larger gain at lr=0.001.

Core experiment (strict discounted objective)	Train	500-episode mean +/- SD
CartPole-v1, b=none	2000	300.240 +/- 97.968
CartPole-v1, b=value	2000	220.218 +/- 45.618
ALE/Pong-v5, b=none	1000	-20.390 +/- 0.827
ALE/Pong-v5, b=value	1000	-20.720 +/- 0.564

Improved Pong extension: lr=0.001 reaches -7.742 +/- 6.311 on selection seed 0. The two independent confirmation seeds average -9.641. Across the three policy means, the average is -9.008 and the sample SD is 1.164. Scores remain negative on average; this is improved learning, not stable mastery.

Objective distinction. Core experiments use the exact start-state discounted objective requested in the handout. Later Pong extensions retain gamma=0.99 inside reward-to-go but remove the outer gamma^t actor weight. This is a different update weighting and is reported explicitly as a supplementary method, not silently substituted for the core objective.

Reading order: sections 1-3 define the model and algorithms; sections 4-7 contain all four required plot pairs; sections 8-13 reconstruct the improvement experiments; sections 14-18 cover substantive validation, edge cases, execution instructions and future work. Appendix A inventories every completed experiment.

1. Shuttle MDP and solution methods

The decision state s is the number of passengers waiting before dispatch: s in {0,...,200}. The action a is 0 (wait) or 1 (dispatch one shuttle). Capacity K=15, dispatch cost c_f=100, holding cost c_h=2, discount gamma=0.95 and independent arrivals A uniformly distributed on {1,2,3,4,5} follow the assignment.

Timing and boundary assumptions

```
left(s,a) = max(s - 15*a, 0)
c(s,a) = 100*a + 2*left(s,a)
s_next = min(200, left(s,a) + A)
```

Dispatch happens before holding cost and arrivals. Overflow above 200 is discarded without an extra penalty. Empty dispatch is permitted but still costs 100. Ties choose wait. These assumptions define a finite-state controlled Markov chain; no service or arrival history beyond the queue is needed.

Finite horizon: enumeration with T=500

```
J_500(s) = 0
J_t(s) = min_a {c(s,a) + 0.95*(1/5)*sum_A J_(t+1)(s_next)}
```

Backward induction enumerates all 201 states, two actions and five arrival outcomes at each time. This produces an optimal feedback policy, including different decisions near the terminal time. The reported time-zero value is J_0(s). Direct enumeration of 2^500 open-loop action strings would be intractable and would not represent every feedback policy; the short-horizon tree validation in section 14 independently checks the Bellman enumeration interpretation.

Infinite horizon: VI and PI

Value iteration starts from zero and applies the discounted Bellman minimum operator until the maximum update residual is below 1e-10. Policy iteration exactly evaluates the current policy with (I-0.95 P_pi)J=c_pi, then greedily improves it until no action changes. Both methods minimize positive costs; a reward formulation would use r=-c and V=-J.

Outcome	Measured value
VI iterations	518
PI iterations	6
Optimal cost J*(0)	576.670167764
Infinite-horizon policy	Wait at 0-12; dispatch at 13-200

2. Required shuttle plots

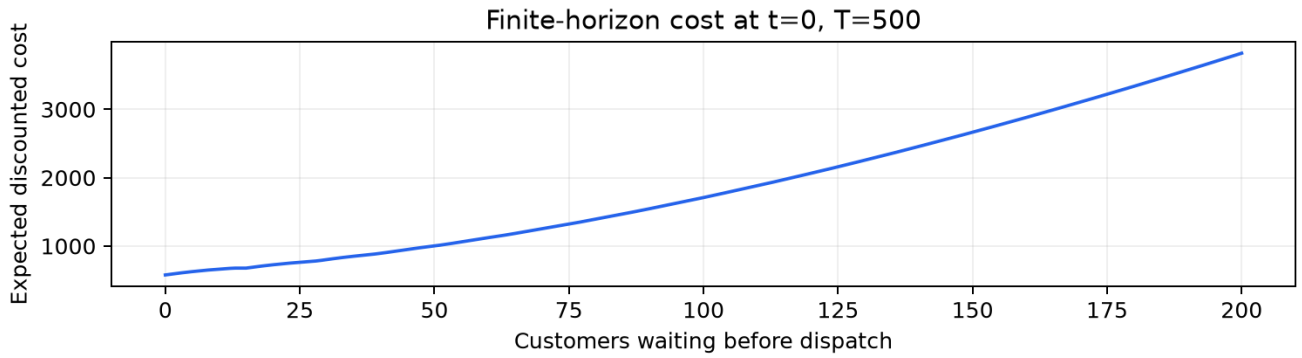


Figure 1. Enumeration / backward induction: optimal positive discounted cost at time 0, horizon T=500.

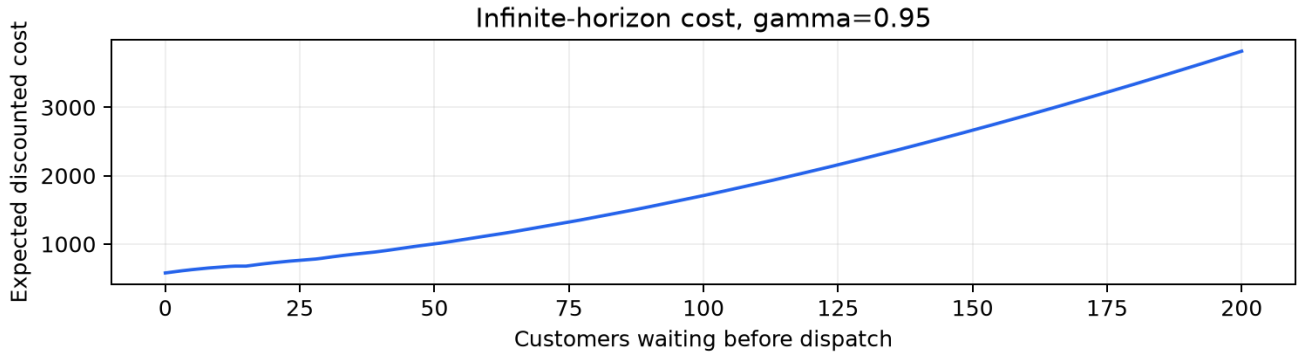


Figure 2. Value iteration: optimal infinite-horizon positive discounted cost.

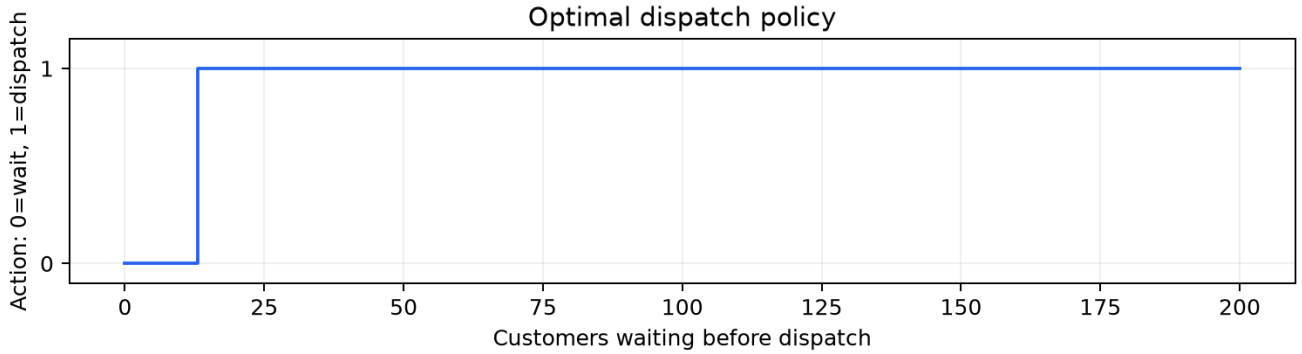


Figure 3. Policy iteration: action 0=wait, action 1=dispatch; threshold 13. All queue states 0-200 are shown.

The threshold can lie below capacity: avoiding another holding period may justify dispatching a partly filled shuttle. At large queues the value curve has service-cycle structure, while the capped queue preserves a bounded state space. The finite T=500 and infinite-horizon values differ by less than 3e-9, but their late-terminal policies are not assumed identical. Raw values are stored in results/shuttle/values.csv.

3. Policy gradient and the value baseline

Core objective and estimator

```
J(theta) = E[sum_t gamma^t * r_t]
G_t = sum_(k=t)^(T-1) gamma^(k-t) * r_k
L_actor = -(1/B) * sum_episodes sum_t gamma^t * log pi_theta(a_t|s_t) * (G_t - b(s_t))
```

Categorical actions are sampled from neural-network logits. PyTorch autograd computes the loss gradient and Adam updates the actor. There is no PPO clipping, reward clipping, return standardization, value bootstrap, or return reset at Pong score points. CartPole uses gamma=0.95; Pong uses gamma=0.99. Evaluation always reports the undiscounted whole-game reward sum, a different metric from the discounted training objective.

Part 2: nonconstant, learned state-value baseline

A separate network b_phi(s) predicts Monte Carlo reward-to-go. Its output is computed before fitting the current trajectory or batch and detached from the actor graph. The critic uses squared prediction error and its own Adam optimizer. The actor therefore cannot improve its loss by differentiating through the baseline. In CartPole the critic is fitted on trajectory chunks; in batched Pong one averaged critic update follows each actor update, with each trajectory normalized by its length.

```
E_a[b(s) * grad log pi(a|s)] = b(s) * grad sum_a pi(a|s) = 0
```

This action-independent baseline leaves the expected score-function estimator unchanged for the chosen weighting. It can reduce variance, but neither a value-function fit nor a lower variance guarantees higher game scores. Direct frozen-policy measurements appear in section 15.

Setting	CartPole core	Pong core
Observation / actor	4 coordinates; 128 tanh units; 2 logits	2 binary 80x80 frames; CNN; 2 logits
CNN (Pong only)	Not applicable	Conv 16: 8x8/stride4; Conv 32: 4x4/stride2; FC128/ReLU
Actor / critic lr	0.001 / 0.001	0.0001 / 0.0003
Episodes / batch	2,000 / 1	1,000 / 8
Optimizer / hardware	Adam; CPU, 1 thread	Adam; Apple MPS, 1 CPU thread

Environment IDs are CartPole-v1 and ALE/Pong-v5, the Gymnasium/ALE registration of the requested Pong-v5. Pong retains the default frameskip=4 and sticky-action probability=0.25. The policy maps its two logits only to actions 2 and 3. It never chooses FIRE or NOOP. Episode history resets on termination or truncation. [R2-R4]

4. Part 1: CartPole without baseline

Configuration: CartPole-v1, baseline=none, seed 0, gamma=0.95, actor Adam lr=0.001. Training: 2,000 episodes. Frozen stochastic-policy evaluation: 500 episodes.

Mean score	Sample SD	Random-policy mean
300.240	97.968	21.906



Figure 4. Training reward and trailing 100-episode mean. Source: results/cartpole_none_lr0001/training.csv.

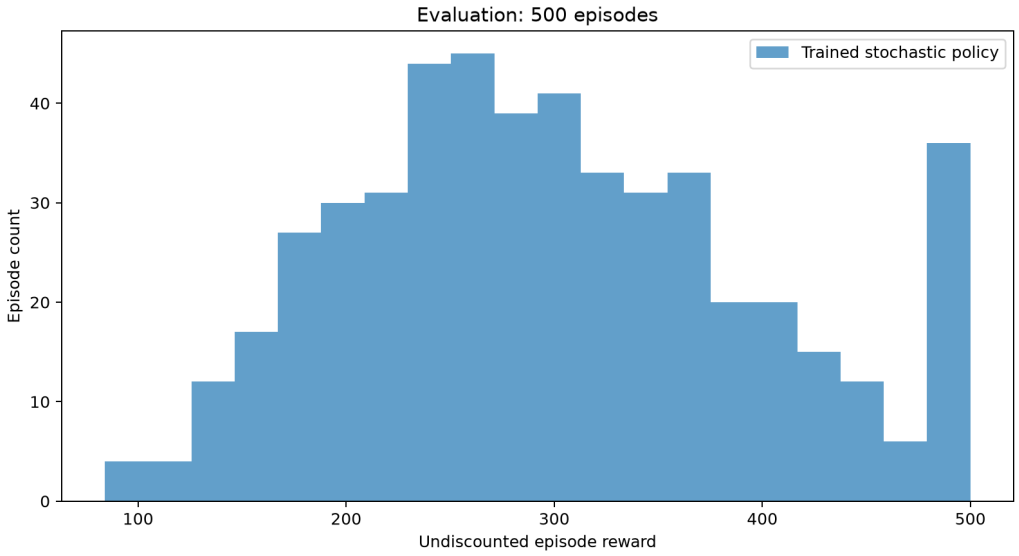


Figure 5. Histogram of the 500 frozen-policy game scores; random-policy reference is displayed separately. Standard deviations use ddof=1.

Reducing actor lr from 0.01 to 0.001 changes the seed-0 evaluation mean from 9.370 to 300.240. The policy is substantially better than random but has not consistently achieved the 500-step limit. It remains stochastic during evaluation.

5. Part 2: CartPole with value baseline

Configuration: CartPole-v1, baseline=value, seed 0, gamma=0.95, actor Adam lr=0.001. Training: 2,000 episodes. Frozen stochastic-policy evaluation: 500 episodes.

Mean score	Sample SD	Random-policy mean
220.218	45.618	21.906



Figure 6. Training reward and trailing 100-episode mean. Source: results/cartpole_value_lr0001/training.csv.

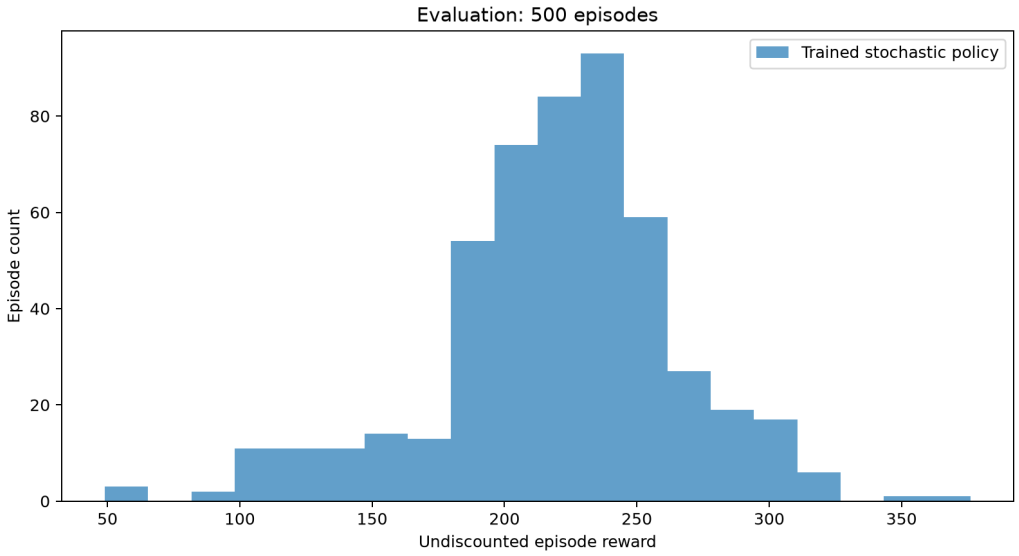


Figure 7. Histogram of the 500 frozen-policy game scores; random-policy reference is displayed separately. Standard deviations use ddof=1.

This matched seed-0 run scores 220.218, below the no-baseline seed-0 result. Across all three matched seeds the baseline has a higher average, illustrating why one favorable run cannot establish a general baseline benefit. Section 15 measures gradient variance directly.

6. Part 1: Pong without baseline

Configuration: ALE/Pong-v5, baseline=none, seed 0, gamma=0.99, actor Adam lr=0.0001. Training: 1,000 episodes. Frozen stochastic-policy evaluation: 500 episodes.

Mean score	Sample SD	Random-policy mean
-20.390	0.827	-20.514

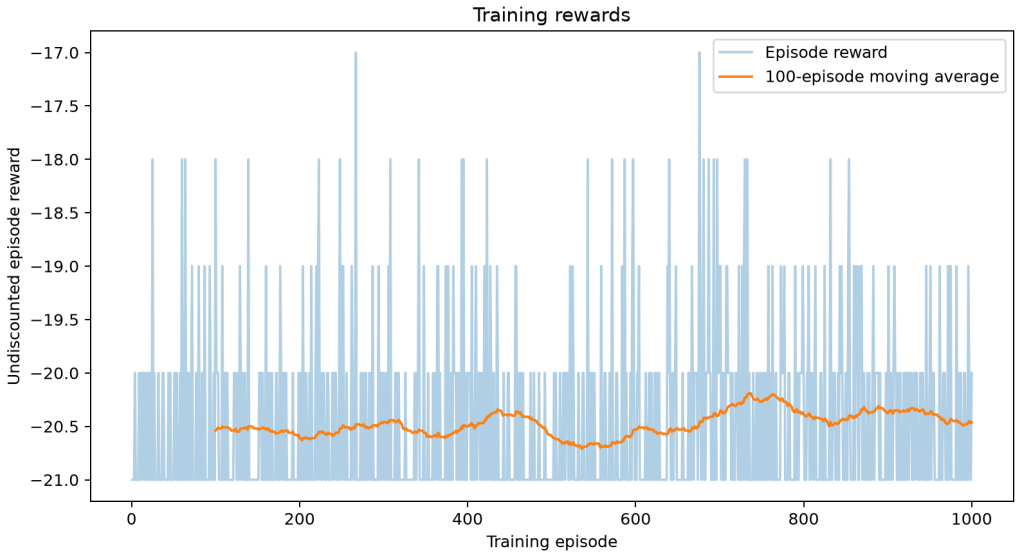


Figure 8. Training reward and trailing 100-episode mean. Source: results/pong_batch_none/training.csv.

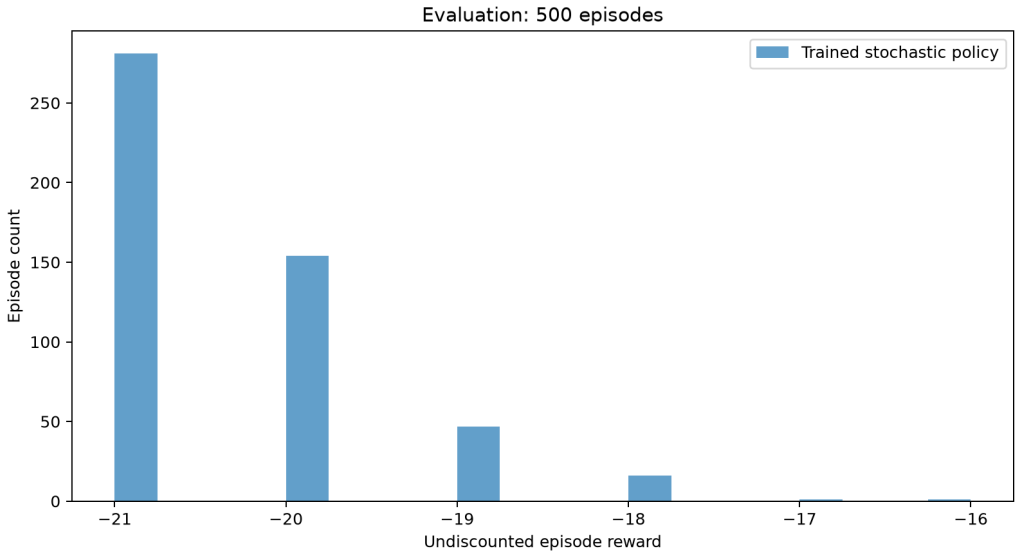


Figure 9. Histogram of the 500 frozen-policy game scores; random-policy reference is displayed separately. Standard deviations use ddof=1.

This strict-objective retry averages eight complete episodes per update, uses actor-head orthogonal initialization with gain 0.01, and clips gradient norm at 5. It still scores near -21. The failure is retained as measured evidence, rather than replacing its histogram with an inferred larger-run outcome.

7. Part 2: Pong with value baseline

Configuration: ALE/Pong-v5, baseline=value, seed 0, gamma=0.99, actor Adam lr=0.0001. Training: 1,000 episodes. Frozen stochastic-policy evaluation: 500 episodes.

Mean score	Sample SD	Random-policy mean
-20.720	0.564	-20.514

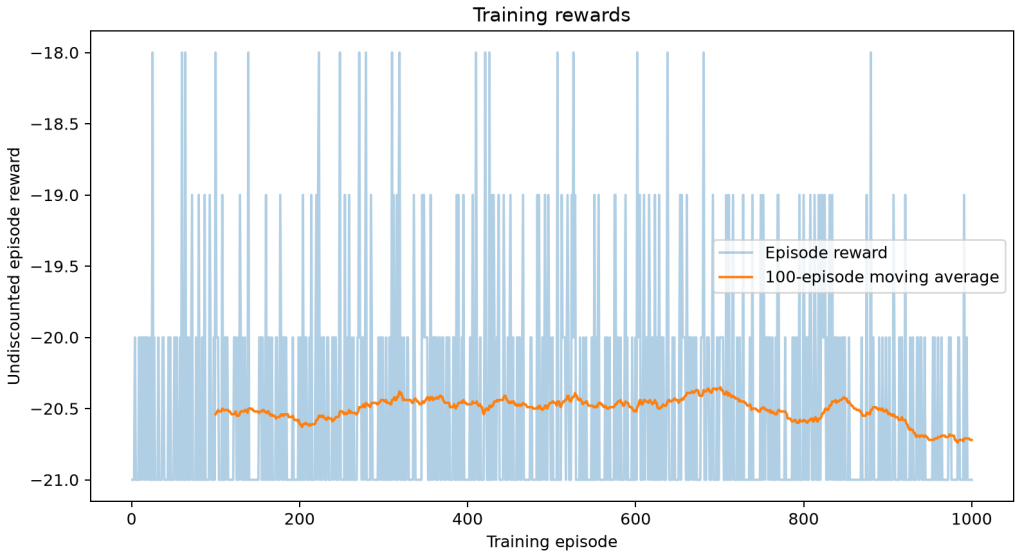


Figure 10. Training reward and trailing 100-episode mean. Source: results/pong_batch_value/training.csv.

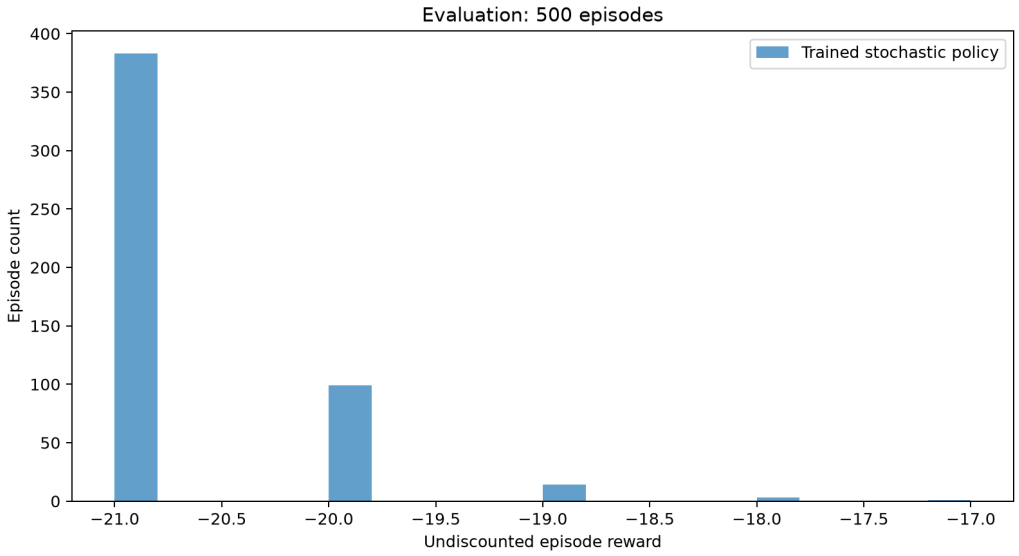


Figure 11. Histogram of the 500 frozen-policy game scores; random-policy reference is displayed separately. Standard deviations use ddof=1.

The independent CNN critic is detached in the actor update and fitted after it. This run is slightly worse than its matched no-baseline counterpart. Later variance tests show reduced estimator variance at this checkpoint, but the score evidence does not establish that the baseline alone improves play.

8. Starting point and CartPole improvements

The first full experiments used the handout Adam actor learning rate 0.01. CartPole trained for 2,000 episodes per baseline and Pong for 200 episodes per baseline. The initial implementation used single-episode REINFORCE and the exact outer discounted weight. Every final policy received 500 evaluation rollouts. Small environment smoke tests were separate and were not treated as assignment results.

Initial experiment	Actor lr	Train	500-episode mean +/- SD
cartpole_none	0.01	2000	9.370 +/- 0.755
cartpole_value	0.01	2000	170.168 +/- 11.184
pong_none	0.01	200	-21.000 +/- 0.000
pong_value	0.01	200	-21.000 +/- 0.000

CartPole without a baseline collapsed below its random reference; Pong reached -21 with zero evaluation variation. These observations motivated parameter changes. For CartPole, lr=0.003 was explored without a baseline and scored 171.432. A matched lr=0.001 comparison was then run at seeds 0, 1 and 2, while preserving the task, architecture and objective.

Training seed	No baseline mean +/- SD	Value baseline mean +/- SD
0	300.240 +/- 97.968	220.218 +/- 45.618
1	92.460 +/- 31.626	362.986 +/- 121.367
2	106.554 +/- 28.962	214.266 +/- 69.583

none: mean of the three policy means = 166.418; between-training-seed sample SD = 116.107.

value: mean of the three policy means = 265.823; between-training-seed sample SD = 84.198.

This supports choosing a smaller CartPole learning rate for the submitted figures, not a claim that one value-baseline run always outperforms one plain run. The evaluation rollouts used during exploration are not an untouched hyperparameter-selection test set.

CartPole task-level sanity check

Deterministic diagnostic (20 episodes each)	Mean reward	Episode ending
Always push left	9.15	Physical termination
Angle + 0.15*angular-velocity feedback	500.00	All 20 reach the 500-step time limit

The predictions are rapid failure under a fixed push and sustained balance under state feedback. Both are observed on separate seeds starting at 9,500,000; reward equals episode length. First-10-step traces retain states, actions and ending flags. This validates state/action conventions, the reward scale and time-limit handling. The controller is hand-coded and is not substituted for a learned-policy score.

9. Pong: stability retry and larger budgets

After the original lr=0.01 failure, the Pong retry jointly lowered actor lr to 0.0001, averaged 8 full trajectories per update, initialized the actor output near an even action distribution and clipped gradients. It retained the CNN, stacked frames, gamma=0.99 and exact outer gamma^t. Because several stabilizers changed together, the initial retry cannot isolate their individual effects.

The next experiment changed only total training budget. Each 1,000-episode actor/critic, Adam state and CPU sampling RNG was copied to an independent directory and resumed to 3,000, then 5,000 episodes. Source CSVs remain exact prefixes; original results were not overwritten. Each endpoint received 500 evaluation episodes.

Total episodes	Actor updates	No baseline mean +/- SD	Value baseline mean +/- SD
1000	125	-20.390 +/- 0.827	-20.720 +/- 0.564
3000	375	-20.952 +/- 0.214	-20.548 +/- 0.693
5000	625	-20.724 +/- 0.563	-20.170 +/- 0.916

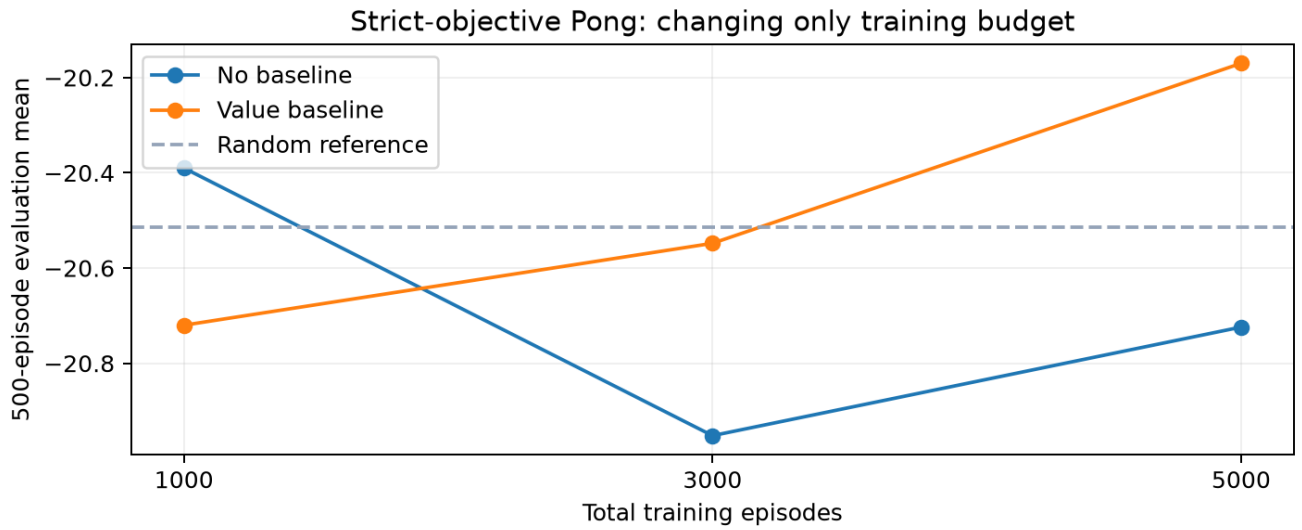


Figure 12. Measured evaluation means at three budgets; each point represents a separately evaluated endpoint, not an inferred score.

At 3,000 episodes the no-baseline result deteriorated; the baseline improved only slightly. At 5,000 both remain poor. Thus this configuration did not become successful merely by extending training. The experiment does not prove that more data can never help; it demonstrates that budget alone was not an effective fix over the tested range. Subsequent experiments hold each fresh-run budget at 3,000 episodes.

10. Pong: the actor time-weighting experiment

The exact start-state discounted objective gives late actions very little weight: 0.99^500 is approximately 0.00657 and 0.99^1000 approximately 0.0000432. This is mathematically appropriate for that objective, but it can conflict with the desire to improve an entire long-game score. It motivated an explicitly labeled alternative weighting, not a claim that the original estimator was a coding error.

```
Core: loss = -sum_t gamma^t * log pi(a_t|s_t) * (G_t - b(s_t))
Extension: loss = -sum_t log pi(a_t|s_t) * (G_t - b(s_t))
```

The extension keeps full-episode gamma=0.99 reward-to-go and removes only the outer time factor. Both branches start from the same 3,000-episode checkpoint and train to 5,000; uniform weighting starts at episode 3,001. Architecture, learning rates, batch, clipping and seed remain fixed. This is a warm-start experiment with inherited optimizer state.

Baseline	Original weight mean	Uniform weight mean	Gain (approx. paired 95% CI)
none	-20.724	-20.444	0.280 [0.194, 0.366]
value	-20.170	-19.954	0.216 [0.090, 0.342]

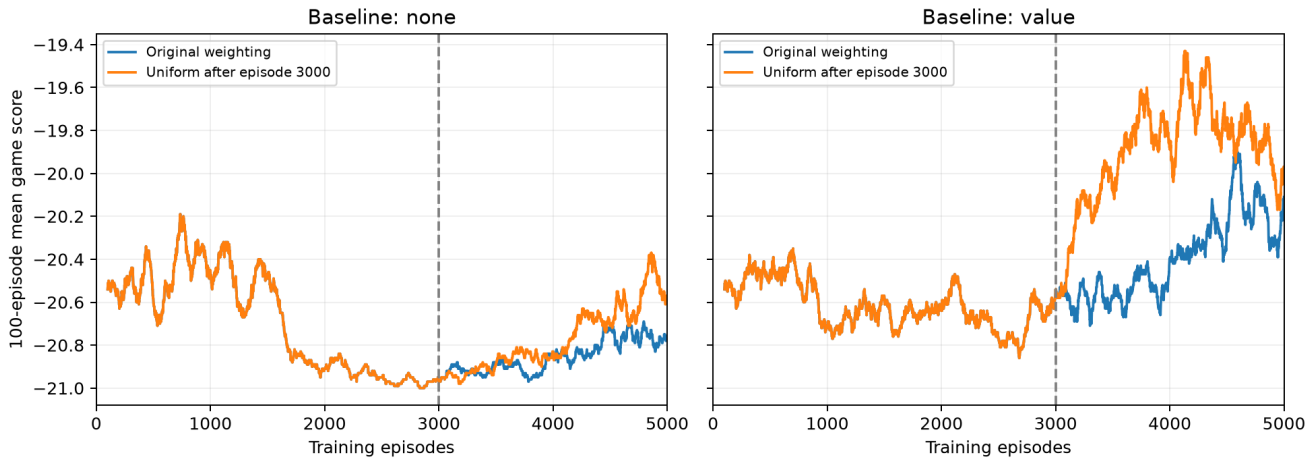


Figure 13. Original and uniform-weight training branches; the vertical line marks the shared 3,000-episode fork.

The small gains are distinguishable from rollout variability for these fixed policies, but neither branch learns effective play. The intervals do not describe variation across training seeds. This practical surrogate is retained for the later exploratory ablations; the strict-objective core results remain separately reported.

11. Pong: motion representation at fixed budget

The stacked-frame CNN must infer motion from two binary images. The alternative explicitly encodes appearing and disappearing pixels in separate binary channels. Subtracting these two channels reconstructs current_frame-previous_frame. Two channels preserve the CNN shape and preserve the sign through packed binary trajectory storage. Directly packing signed -1/0/1 differences would lose that sign.

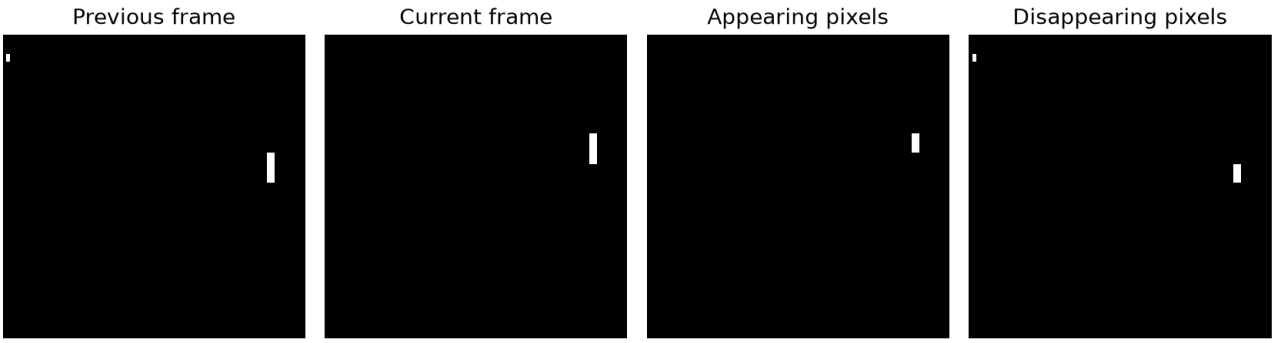


Figure 14. Real ALE observations and the two motion channels. Static pixels disappear from both motion channels.

A fair comparison starts both value-baseline models fresh: stacked frames versus split frame difference, actor lr=0.0001, critic lr=0.0003, uniform time weighting from episode 1, seed 0, 3,000 episodes and 500 evaluations. Reusing a stacked-frame checkpoint with the new encoding is rejected by the program.

Input	500-episode mean +/- SD
Stacked frames	-20.010 +/- 1.217
Split frame difference	-19.888 +/- 1.038

Motion minus stacked score = 0.122; approximate paired 95% interval [-0.018, 0.262]. The interval includes zero. This experiment does not establish a reliable advantage of motion input at this learning rate.

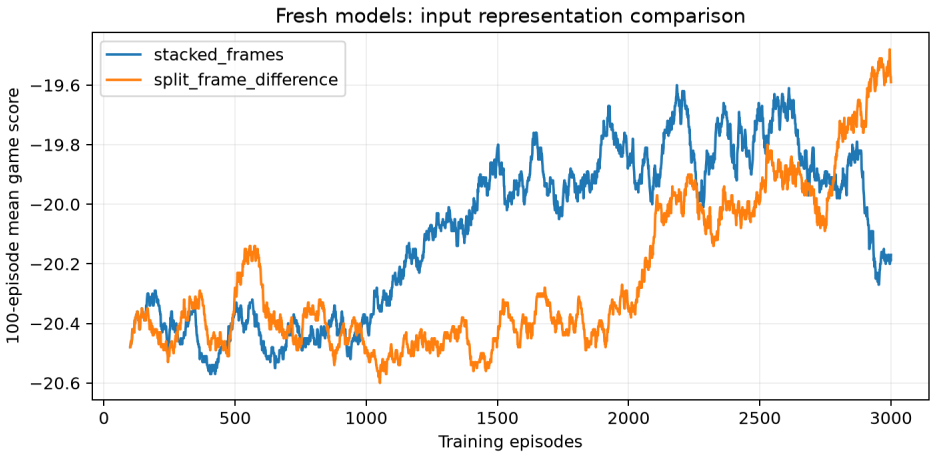


Figure 15. Fresh models under matched budgets. A visually higher training segment is not a substitute for the independent score comparison.

12. Pong: actor learning rate is the larger gain

Because the representation experiment did not resolve weak learning, the next comparison fixed split-frame-difference inputs, the CNN, value baseline, uniform time weighting, critic lr=0.0003, batch=8, clipping=5 and seed 0. The completed fresh lr=0.0001 model was reused unchanged; fresh actors and critics were trained at actor lr=0.0003 and 0.001. Each used 3,000 episodes, exactly 375 actor updates, followed by 500 evaluation episodes.

Actor lr	Evaluation mean +/- SD	Gain over 0.0001
0.0001	-19.888 +/- 1.038	Reference
0.0003	-19.122 +/- 1.407	0.766
0.001	-7.742 +/- 6.311	12.146

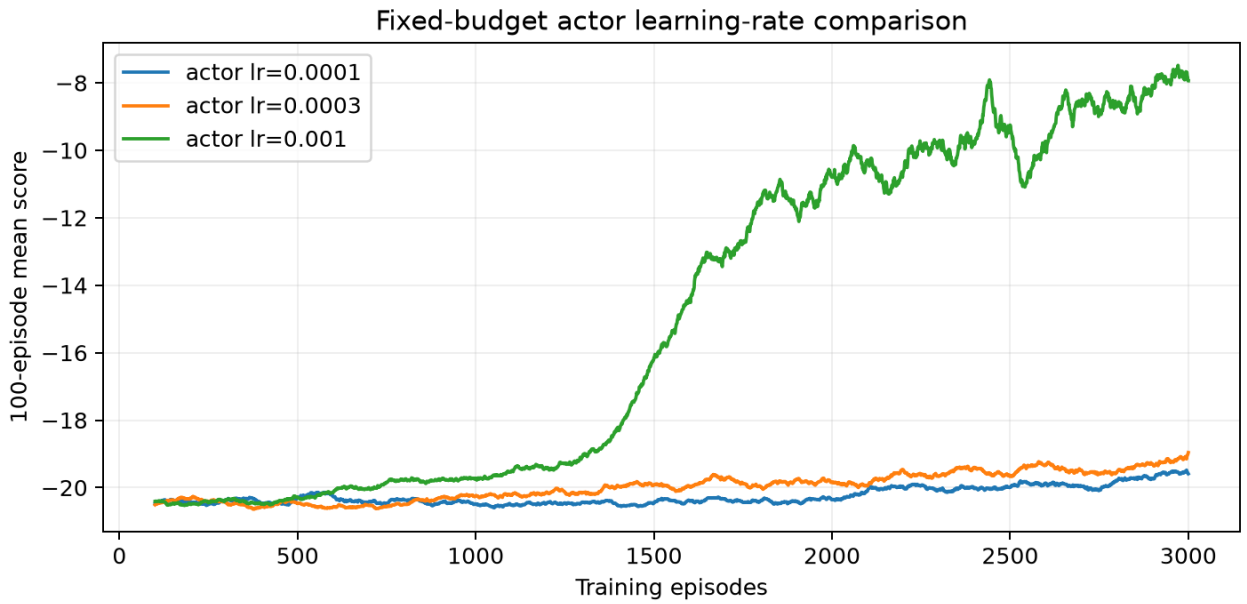


Figure 16. Matched training curves. The actor lr=0.001 configuration learns much faster within the fixed 3,000-episode budget.

The lr=0.001 gain is 12.146 score units, with approximate paired rollout interval [11.585, 12.707]. This is much larger than the earlier budget, weighting and representation gains. It is evidence for the tested actor learning rate within this specific practical configuration, not proof of its effect under the strict outer-discounted objective or every network.

The evaluation data here were used to select lr=0.001. An independent confirmation is therefore needed before presenting the selected seed as representative; section 13 provides that check. A new no-baseline counterpart under this improved configuration has not been run, so its improvement is not attributed to the baseline alone.

12a. Improved Pong extension: selected seed 0

Configuration: ALE/Pong-v5, baseline=value, seed 0, gamma=0.99, actor Adam lr=0.001. Training: 3,000 episodes. Frozen stochastic-policy evaluation: 500 episodes.

Mean score	Sample SD	Random-policy mean
-7.742	6.311	-20.514

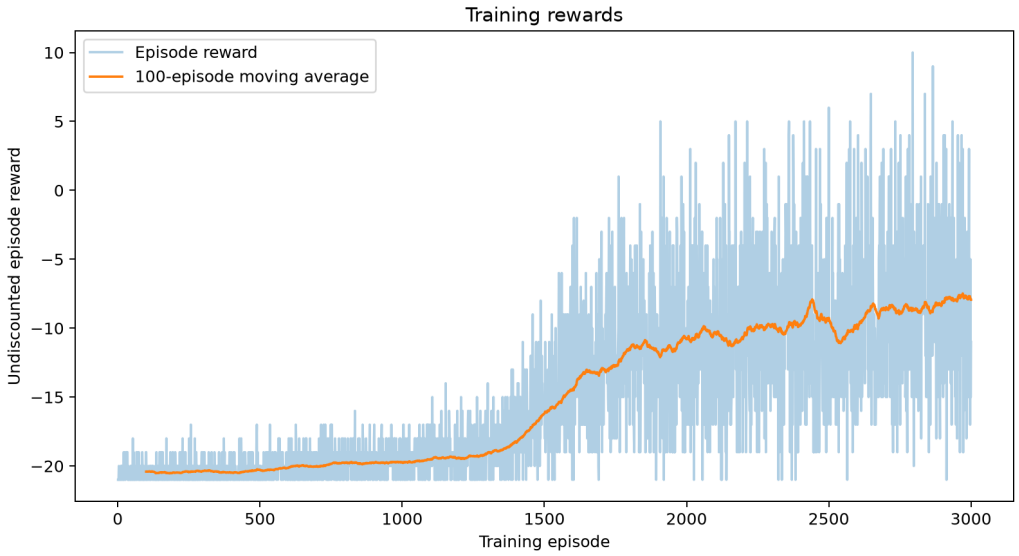


Figure 17. Training reward and trailing 100-episode mean. Source: results/pong_motion_lr001_value/training.csv.

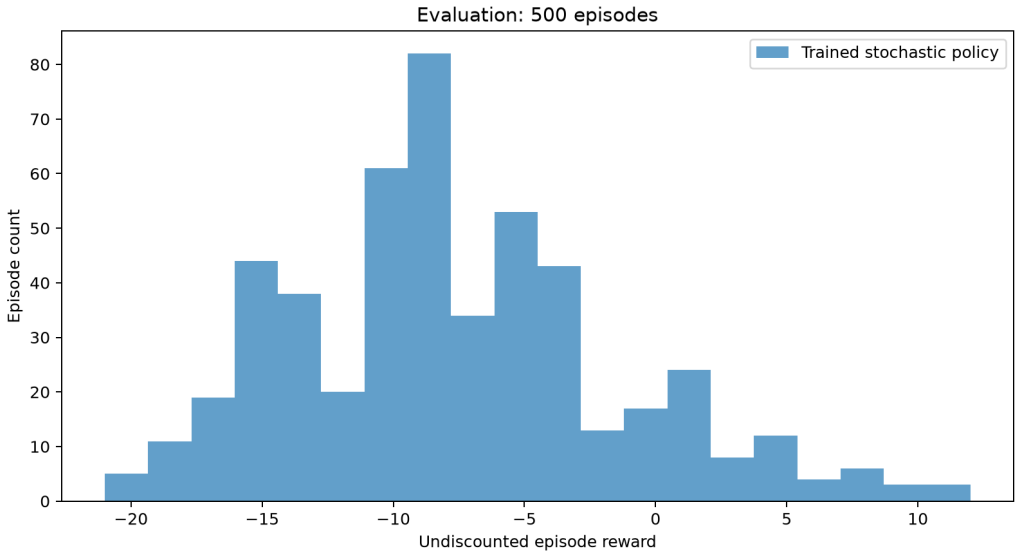


Figure 18. Histogram of the 500 frozen-policy game scores; random-policy reference is displayed separately. Standard deviations use ddof=1.

Supplementary uniform-time-weighted method, not the strict-objective core experiment. The selected policy scores -7.742 +/- 6.311; 60 of 500 evaluation scores are positive, and the maximum is +12. The mean remains negative, so the policy is not claimed to win reliably. Selection on these results makes independent seeds important.

13. Independent training-seed confirmation

The learning rate was selected on seed 0. Two additional models were then initialized and trained independently at seeds 1 and 2, with unchanged hyperparameters and the same 3,000-episode budget. No optimizer or trained representation is transferred. The program seed changes initialization, sampling and disjoint environment-seed ranges; each model receives 500 evaluations and a seed-matched random reference.

Training seed	Role	Mean +/- rollout SD	Positive scores / 500
0	Selection	-7.742 +/- 6.311	60
1	Confirmation	-9.250 +/- 3.668	9
2	Confirmation	-10.032 +/- 3.796	2

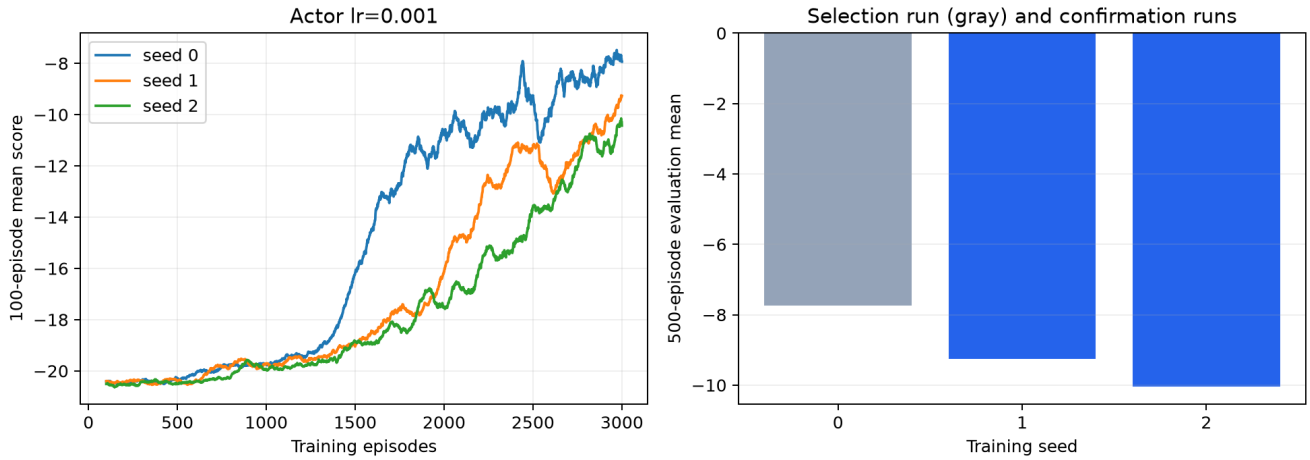


Figure 19. Individual training curves and independent 500-episode score means. Seed 0 was used for hyperparameter selection.

The independent seed-1/seed-2 mean is -9.641. The three-seed mean is -9.008; the sample SD between the three policy means is 1.164. These describe training-procedure variability separately from each policy's rollout SD.

The conclusion is limited: both confirmation runs improve substantially over their random references, supporting repeatable learning of the selected configuration. Only three training seeds are available, and average scores remain negative. No new low-learning-rate model was trained at seeds 1 and 2, so this is not a complete learning-rate-by-seed interaction study. Pooling 1,500 scores would not create 1,500 independent training runs.

Training seed ranges: 20,000,000 + 100,000*seed through that value +2,999. Evaluation ranges: 4,000,000 + 10,000*seed through that value +499. Training, evaluation and separate diagnostic ranges are disjoint for these reported runs.

14. Shuttle validation beyond basic unit tests

The handout asks for predicted algorithm behavior and quantitative evidence. The following tests compare different algorithms, independently enumerate small problems, change meaningful parameters, and compare DP with independent simulation. They are not merely file-format checks.

Prediction / test	Observed result
Tiny problem: 5 states, 2 arrivals, horizon 4; recursive action/outcome tree vs DP	Maximum error 0.0
VI and PI must agree	Value max error 1.965e-09; 0 policy mismatches
PI must satisfy Bellman optimality	Residual 9.095e-13
T=500 approximates discounted infinite horizon	Value max error 2.979e-09
Free dispatch should serve positive queues	All positive states dispatch
Zero capacity should make paid dispatch useless	No state dispatches
gamma=0 should remove future effects	All values equal minimum immediate cost

Independent Monte Carlo cost validation

Initial queue	DP cost	Simulation mean	Standard error
0	576.670	576.769	0.156
12	672.892	673.024	0.153
13	676.670	676.848	0.157
15	676.670	676.286	0.157
200	3821.586	3821.739	0.375

For each queue, 40,000 independent 500-step simulations use seed 314159. Four of five values fall in marginal 95% intervals; state 15 differs by -2.44 standard errors and is retained. All five fall in the simultaneous 95% Bonferroni intervals. The maximum omitted discounted tail is below 6.9e-8. Agreement within quantified simulation error supports the transition/cost implementation; it does not validate arbitrary overflow assumptions.

```
python src/shuttle.py --out runs/shuttle
python src/validate_shuttle_simulation.py
```

15. Policy-gradient validation and diagnostics

Independent gradients and baseline variance

A two-step Bernoulli control problem enumerates all four action sequences and compares the analytic discounted-objective derivative with the expected score-function estimator. Both zero and nonzero constant baselines have error below 6e-8. A CartPole-network parameter also matches a central finite difference to about 4.2e-12. These checks establish the implemented estimator on a problem with a known derivative, not merely that training executes.

Frozen actor / held-out samples	Plain variance trace	Baseline trace	Ratio (95% bootstrap interval)
cartpole, 64 trajectories	21499	33.83	0.00157 [0.00129, 0.00191]
pong, 16 trajectories	2326.8	281.96	0.12118 [0.07340, 0.29105]

Variance is the trace of the sample covariance of full actor gradients. Both estimators use exactly the same held-out trajectories and the same frozen actor; the learned critic is frozen too. Intervals use 300 paired bootstrap resamples. CartPole uses the final 2,000-episode baseline checkpoint and Pong the strict 1,000-episode checkpoint. These measurements do not establish variance reduction for every later method or prove improved game score. The old almost-deterministic Pong run had numerically degenerate variance and is excluded from meaningful ratio interpretation.

Problem-specific implementation and environment checks

The chunked actor/critic loss is checked against an independent dense loss, with and without baseline, under both time weights; the synthetic trajectory crosses the 256-step chunk boundary. Tests also verify critic detachment. Motion tests check sign reconstruction, packed-state round trips, static-pixel cancellation, history reset and agreement between the state used to sample actions and the state later used for the gradient.

Real ALE action check	Observed player-paddle response
Action 2, three recorded step intervals	Center y: 34.5 -> 11.5 (upward)
Action 3, three recorded step intervals	Center y: 34.5 -> 57.5 (downward)
Separate 10-episode diagnostic	Hand-coded tracking mean -16.1; random mean -20.3

These probes confirm movement direction in real frames. The simple tracking controller is weak and is not a trained-policy result. Activation checks on the committed 3,000-episode models found nonzero ReLU activity; the reproducible measurements are in results/pong_endpoint_activations.json. These are network-health checks, not score evidence. Original first-10-step traces remain available for the initial PG runs.

```
python src/policy_gradient.py --test-only
python src/reproduce_validation.py
python -m unittest discover -s src -p "test_pong_*.py"
python src/diagnose_cartpole_control.py
python src/diagnose_pong_control.py
python src/diagnose_pong_activations.py
python src/audit_results.py
```

16. Edge cases, objective checks and limits

Part / edge case	Handling and evidence
Shuttle: s=0, empty dispatch	Never create negative passengers; dispatch still costs 100. Boundary test checks the immediate cost.
Shuttle: s=200 and overflow	Waiting yields 200 for every arrival; dispatch yields 186-190. Clipping and overflow assumption are explicit.
Shuttle: terminal time and ties	J_500=0; late-horizon actions can differ. Greedy ties choose wait consistently.
CartPole: termination or truncation	Both end the sampled episode; no value bootstrap is added. The finite 500-step return is evaluated.
Pong: restricted actions and resets	Only actions 2/3; two-frame history resets per episode. Split motion starts with zero change.
Sparse rewards / long episodes	Full-episode returns, no score-point reset. Exact gamma^t and uniform variants are explicitly distinguished.
Variable trajectory lengths / memory	256-step gradient chunks preserve the sum; Pong averages per episode, not by pooling all steps.
Non-finite or extreme gradients	Reject non-finite gradients; batched Pong clips norm at 5. Low-entropy/zero-variance failures are retained.
Warm-start continuation	Check episode count against CSV; preserve source prefix, optimizer and CPU sampler RNG; record source hashes.
Changing input representation	Reject cross-representation resume; use a fresh network and optimizer for fair comparisons.
Baseline fitting leakage	Detach baseline; compute before fitting the current data. Direct variance checks use frozen actor/critic.

Training objective and whole-game score can disagree

Strict-objective policy	Mean discounted return	Mean score
random	-1.4135	-20.415
none	-1.2554	-20.410
value	-1.2424	-20.740

This separate diagnostic uses 200 episodes per policy on held-out seeds starting at 6,000,000. Both trained strict-objective policies improve discounted return over random while remaining poor at whole-game score. This demonstrates why the two metrics must be reported separately; it does not identify exactly which reward timings changed. Run python src/evaluate_pong_objective.py --episodes 200 --device cpu to reproduce.

17. Reproduction, evidence and submission

Use Python 3.12. The measured game runs used PyTorch 2.14.1, Gymnasium 1.4.0, ALE 0.12.1, NumPy 2.5.4, Adam and one CPU thread. CPU was used for CartPole and historical runs, Apple MPS for batched Pong. The recorded lock file describes the actual training environment. Report rendering is separate from training; hardware and numeric differences can change stochastic trajectories.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-lock.txt
```

From the repository root, write repeats to fresh runs/ directories. The four commands below correspond exactly to the core report configurations. Every trained policy is evaluated for 500 episodes; --device cpu is available if MPS is unavailable.

```
python src/policy_gradient.py --game cartpole --baseline none --lr 0.001 --episodes 2000 --eval-episodes \
500 --out runs/cartpole_none
python src/policy_gradient.py --game cartpole --baseline value --lr 0.001 --episodes 2000 --eval-episodes \
500 --out runs/cartpole_value
python src/pong_batch.py --baseline none --lr 0.0001 --episodes 1000 --batch-episodes 8 --eval-episodes \
500 --device mps --out runs/pong_none
python src/pong_batch.py --baseline value --lr 0.0001 --episodes 1000 --batch-episodes 8 --eval-episodes \
500 --device mps --out runs/pong_value
```

The improved seed-0 extension can be repeated without changing source code. For independent confirmation change --seed to 1 or 2 and choose fresh output directories.

```
python src/pong_batch.py --baseline value --lr 0.001 --value-lr 0.0003 --episodes 3000 --eval-episodes \
500 --batch-episodes 8 --seed 0 --device mps --time-weighting uniform --representation \
split_frame_difference --out runs/pong_improved_seed0
```

The batch scripts run_learning_rate_comparison.py and run_seed_validation.py preserve completed results and reject duplicate manifests. They are one-time experiment controllers, not report-regeneration commands. For a repeat, use the explicit training command with fresh outputs. Exact historical source snapshots and SHA-256 manifests remain in results/. Atari ROMs and virtual environments are not committed.

```
python src/audit_results.py
python src/reproduce_validation.py
python src/prepare_report_figures.py
python src/build_report.py
```

No retraining is needed to regenerate this PDF or repeat the included actor/critic gradient-variance validation capsules. reports/report_config.json selects the core experiments. results/report_evidence.json ties every reported experiment to raw CSV and summary hashes. The report and required plots are in the public repository. Submit reports/assignment1_report.pdf to Canvas; updating GitHub does not itself submit Canvas.

18. What three additional months would add

Period	Predictions and experiments	Evidence to collect
Month 1: isolate causes	Run a no-baseline counterpart to the improved Pong family at matched seeds. Cross strict/uniform weights with actor lr; test constant vs learned baseline. Vary shuttle capacity, cost ratios, queue cap and overflow rule.	Paired seed effects, discounted and undiscounted returns; Bellman residuals; simulated costs and threshold changes.
Month 2: quantify robustness	Use at least 10 predeclared training seeds, separate tuning and confirmation seeds, and equal step/update budgets. Test observation noise and initial-state shifts. Study altered Atari sticky actions only as explicitly separate tasks.	Between-seed uncertainty and failure rates; held-out evaluation curves; robustness profiles without conflating environment changes.
Month 3: understand learning	Freeze early/mid/late checkpoints to measure baseline gradient variance. Compare architectures and motion encodings. Study nonstationary/heavy-tailed shuttle arrivals. Replicate selected models across CPU/MPS.	Full-gradient covariance traces and baseline calibration; sample and wall-clock efficiency; independent replication and model-assumption sensitivity.

For shuttle sensitivity, the prediction is that increasing dispatch cost tends to delay dispatch, while increasing holding cost tends to accelerate it. Threshold changes and boundary exceptions would be measured rather than assumed. For baselines, predicted variance reduction would be tested directly at each checkpoint, separately from changes in game score.

Conclusions and remaining limitations

The shuttle solution is supported by independent small-tree enumeration, Bellman consistency, controlled parameter changes and large independent simulations. For policy gradients, lower variance is verified directly rather than inferred from smoother rewards. The original Pong failures remain visible; extra training alone and the first motion ablation were not sufficient. A larger actor learning rate in the practical motion-input family produced the largest tested score gain, and independent seeds support repeatable learning. Average Pong performance remains below winning.

No trained result is extrapolated from a demo or an unrun budget. Exploratory choices and selected seed 0 are distinguished from independent confirmation. Three training seeds are not enough for a broad reliability claim. The improved baseline family lacks a matched no-baseline counterpart, and the uniform surrogate is not the exact start-state objective of the core experiments.

Sources and assistance

[R1] Course handout, mlds_assignment1.pdf, pages 1-4. Supplied assignment, due 26 October 2026 at 11:59 pm.

[R2] Farama Foundation. CartPole-v1 environment specification.
https://gymnasium.farama.org/environments/classic_control/cart_pole/

[R3] Farama Foundation. ALE Pong environment and action specification.
https://ale.farama.org/environments/pong/

[R4] OpenAI Spinning Up. Intro to Policy Optimization: reward-to-go and baselines.
https://spinningup.openai.com/en/latest/spinningup/rl_intro3.html

AI assistance was used for implementation, debugging and report preparation. All numerical experiment results are from actual executions and retained artifacts. The author should be able to explain the estimators, assumptions and validation evidence independently.

Appendix A. Complete measurement inventory

All formal training experiments below have 500 final-policy evaluation episodes. The explicitly named early diagnostic uses only 20 training and 2 evaluation episodes and is not an assignment-sized result. Other short smoke demos are kept separately. Sample SD is over evaluation episodes, not training seeds. Raw curves and histograms are in each result directory; 100-episode averages apply when enough training episodes exist.

Result directory (under results/)	Train	Eval	Mean	Sample SD
cartpole_none	2000	500	9.370	0.755
cartpole_none_lr0001	2000	500	300.240	97.968
cartpole_none_lr0001_seed1	2000	500	92.460	31.626
cartpole_none_lr0001_seed2	2000	500	106.554	28.962
cartpole_none_lr0003	2000	500	171.432	42.960
cartpole_value	2000	500	170.168	11.184
cartpole_value_lr0001	2000	500	220.218	45.618
cartpole_value_lr0001_seed1	2000	500	362.986	121.367
cartpole_value_lr0001_seed2	2000	500	214.266	69.583
pong_batch_none	1000	500	-20.390	0.827
pong_batch_value	1000	500	-20.720	0.564
pong_budget3000_none	3000	500	-20.952	0.214
pong_budget3000_value	3000	500	-20.548	0.693
pong_budget5000_none	5000	500	-20.724	0.563
pong_budget5000_value	5000	500	-20.170	0.916
pong_motion_lr0003_value	3000	500	-19.122	1.407
pong_motion_lr001_value	3000	500	-7.742	6.311
pong_motion_lr001_value_seed1	3000	500	-9.250	3.668
pong_motion_lr001_value_seed2	3000	500	-10.032	3.796
pong_none	200	500	-21.000	0.000
pong_repr_motion_value	3000	500	-19.888	1.038
pong_repr_stack_value	3000	500	-20.010	1.217
pong_uniform5000_none	5000	500	-20.444	0.798
pong_uniform5000_value	5000	500	-19.954	1.141
pong_value	200	500	-21.000	0.000
pong_value_early_diagnostic	20	2	-20.500	0.707

Requirement map: shuttle MDP/assumptions and three requested figures - sections 1-2; Part 1 and Part 2 algorithms/baseline - section 3; all four required reward curves and 500-episode histograms - sections 4-7; failure-driven improvements - sections 8-13; quantitative validation - sections 14-15; per-part edge cases - section 16; reproducibility/GitHub/Canvas - section 17; three additional months - section 18.